title: ‘Game Architecture’ project: ‘Proyecto-Movimiento’ date: ‘2026-04-16’ author: ‘Jpinzon’ version: ‘1.0’ stepsCompleted: [1, 2, 3, 4, 5, 6, 7, 8, 9] status: ‘complete’ engine: ‘Unity 6.3 LTS’ platform: ‘Android’

Source Documents

gdd: ‘c:\Users\jpinzon\Proyecto-Movimiento_bmad-output\planning-artifacts\gdd-slot-car-racing-ar.md’ epics: null brief: null ux: ‘c:\Users\jpinzon\Proyecto-Movimiento_bmad-output\planning-artifacts\ux-design-specification.md’

Game Architecture

Executive Summary

Slot Car Racing AR architecture is designed for **Unity 6.3 LTS** targeting **Android**.

Key Architectural Decisions:

- URP 17.6, AR Foundation 6.5, ARCore XR Plugin 6.5, Netcode for GameObjects 2.11 y Unity Transport 6.5 forman un stack coherente para AR movil y multijugador local.
- El juego usa una topologia host-authoritative con `TrackFreeze`, `Race Space Snapshot` y `Spatial Trust` para proteger la idea central de “misma pista, misma carrera”.
- La estructura hibrida, los concerns transversales y los patrones de implementacion reducen ambiguedad para futuros agentes y para implementacion en Unity.

Project Structure: organizacion hibrida con 8 sistemas tecnicos principales y limites explicitos entre `Core`, `Runtime`, `UI`, `Debug` y tooling de editor.

Implementation Patterns: 7 patrones definidos para asegurar consistencia entre agentes y evitar decisiones divergentes en calibracion, sincronizacion espacial, comunicacion, estado y acceso a datos.

Ready for: Epic implementation phase

Document Status

This architecture document was created through the GDS Architecture Workflow and is ready to guide implementation.

Steps Completed: 9 of 9 (Architecture Complete)

Project Context

Game Overview

Slot Car Racing AR - juego movil de carreras en realidad aumentada para Android donde dos jugadores comparten una pista miniatura anclada sobre una mesa real mediante un marcador primario compartido y marcadores secundarios de borde usados en la calibracion de superficie, y compiten con un unico control de aceleracion basado en timing.

La experiencia tecnica no depende de sincronizar un mundo AR completo por red, sino de combinar dos capas distintas:

- una referencia espacial local basada en marcador fisico compartido
- una verdad de carrera sincronizada por red local con autoridad del host

Technical Scope

Platform: Android
Genre: AR local multiplayer racing game
Project Level: High

Core Systems

System	Complexity	GDD Reference
AR marker tracking and spatial anchoring	High	11.3, 17.1
Shared spatial calibration and re-alignment	High	6.2, 9.5, 17.2
Local session orchestration and lifecycle	High	6.2, 9.1, 9.5, 11.4
Host-authoritative networking and sync contract	High	7.7, 11.4
Rail/spline-based race simulation	Medium	7.1-7.6, 11.5
Camera-first AR HUD and UI state system	Medium	9.2-9.5, UX responsive/accessibility
Recovery, degraded mode, and recovery state machine	High	9.5, 17, 18
Runtime telemetry and performance governance	Medium	11.6, 18, UX step 13

Technical Requirements

- Android-only MVP with ARCore-compatible phones
- Unity-based implementation as primary direction
- Two-player required local multiplayer
- One primary shared marker as spatial anchor plus optional secondary boundary markers during setup
- One predefined track and one quick-race mode
- Host-client topology with host-authoritative race state
- Client sends input, host resolves simulation, both devices use the same calibrated race space derived from a shared marker frame
- Landscape gameplay as the only gameplay mode
- Setup under 2 minutes and join flow under 30 seconds
- Stable race session with rematch support and failure recovery
- Performance floor for shipping: stable 30 FPS minimum on target device
- Experience target for hot path: strive for smoother race flow closer to 60 FPS where feasible, without making it a blocking MVP promise
- Protected central AR zone with camera-first HUD rules
- No raw AR world synchronization over network; network syncs race state, marker syncs shared reference

Complexity Drivers

- Shared spatial perception without raw AR world synchronization
- Perceptual alignment between devices, not just logical alignment
- Tight coupling between AR tracking state, network state, and UX readiness state
- Re-calibration and re-acquisition flows without forcing full restart
- Camera-first HUD rules with safe-area, touch-zone, and visibility constraints
- Cross-device AR consistency under different angles, lighting, and thermal conditions
- Separation between simulation truth and AR presentation truth
- Need for explicit technical contracts before engine/starter selection

Technical Contracts Needed Before Engine Selection

- Exact AR stack decision:
  - AR Foundation + ARCore plugin/extensions
  - or alternative native approach
- Exact network stack decision:
  - Netcode for GameObjects + Unity Transport, or alternative
- Session authority contract:
  - what the host owns
  - what the client predicts
  - what gets corrected
  - how reconciliation is smoothed
- Shared anchoring contract:
  - marker rules
  - re-anchor rules
  - rescan behavior
  - playable tolerance before forcing reset
- Failure contract:
  - recoverable vs non-recoverable states
  - host loss behavior
  - reconnect/rejoin rules
  - behavior on background/foreground transitions
- Performance contract:
  - FPS target and floor
  - frame-time budget
  - thermal tolerance
  - device support tiers
- Testability contract:
  - measurable NFRs
  - success gates for prototype readiness
  - minimum device and lighting matrix

Technical Risks

- Tracking instability under real lighting conditions
- Perceived desynchronization between devices even if race state is technically correct
- Spatial drift or divergence in how the same marker is interpreted by each device
- Local network instability on hotspot or shared Wi-Fi
- Thermal throttling and sustained performance degradation
- Hidden complexity in recovery flows for host loss, tracking loss, and resync
- Premature package/tooling lock-in without version policy
- Scope creep at the technical level even if gameplay scope stays small

Failure States Requiring Architectural Treatment

- Only one player loses tracking while the other remains stable
- Both players track the marker but perceive different scale/orientation of the same track
- Host loses network, minimizes app, or loses tracking mid-session
- Client disconnects briefly during active race
- Marker leaves field of view during sustained acceleration input
- Device heat degrades FPS, then tracking, then network behavior
- Session can no longer be considered spatially trustworthy even if logic still runs

Initial Non-Functional Requirements To Quantify In Architecture

- Maximum acceptable setup time
- Maximum acceptable marker reacquisition time
- Maximum visible race-state desync tolerance
- Minimum stable FPS on reference hardware
- Maximum tolerable thermal degradation during repeated sessions
- Minimum session success rate in real tabletop conditions
- Maximum rate of unrecoverable session failures
- Minimum HUD legibility and touch accuracy constraints already defined in UX

Delivery Implications

This project is not architecturally difficult because of content scale. It is difficult because three realities must agree at the same time:

- what the camera sees
- what the network says
- what the player believes is happening

That makes the project high complexity for MVP architecture, even though the gameplay loop itself is intentionally narrow.

Engine & Framework

Selected Engine

Unity v6.3 LTS

Rationale:
Unity es el mejor ajuste para este proyecto por la combinacion especifica de requisitos: Android, AR movil, marcador fisico compartido, UI camera-first y networking local host-client. Frente a Godot y Unreal, Unity reduce riesgo en AR movil y ofrece un camino mas directo para integrar AR Foundation, ARCore y un stack de multijugador local sin inflar innecesariamente el costo tecnico del MVP.

La seleccion se considera valida bajo una condicion arquitectonica explicita: debe superarse una validacion temprana en dispositivos reales para confirmar compatibilidad de paquetes, estabilidad de tracking y comportamiento termico aceptable.

Project Initialization

Se recomienda iniciar **desde un proyecto limpio**, no desde un template de aprendizaje.

Starter elegido: proyecto nuevo de Unity creado desde Unity Hub con Unity 6.3 LTS.

Decision de arranque:
Limpio si, pero no vacio en la practica. El proyecto debe nacer con un scaffold tecnico minimo desde el primer commit.

Scaffold tecnico minimo obligatorio

- configuracion base de Android desde el inicio
- escena vertical slice minima con:
  - marcador detectado
  - pista visible
  - un coche funcional
  - boton de aceleracion
- instrumentacion inicial para:
  - FPS
  - tracking loss
  - GC spikes
  - latencia/red cuando aplique
- estructura base preparada para crecimiento sin depender de templates de muestra

No seleccionado:
Template oficial `Get Started With Unity`, porque esta pensado para onboarding y tutoriales en editor, no para un MVP AR multijugador con restricciones de rendimiento y paquetes sensibles.

Engine-Provided Architecture

Component	Solution	Notes
Rendering	Unity rendering stack	Unity provee la base de render 3D movil; la eleccion exacta de pipeline queda abierta
Physics	Unity 3D Physics	Disponible, aunque la simulacion principal del MVP debe seguir siendo parametrica sobre rail/spline
Audio	Unity Audio system	Suficiente para countdown, penalizacion, meta y feedback corto
Input	Unity input stack	El proyecto usara input tactil simple; la implementacion exacta se define despues
Scene Management	Unity scene-based architecture	Adecuado para separar bootstrap, lobby, carrera y pruebas
Build System	Unity Android build pipeline	Compatible con Android target y modulos moviles desde Unity Hub

Remaining Architectural Decisions

The following decisions must be made explicitly:

- stack AR exacto:
  - AR Foundation
  - ARCore XR Plugin
  - definir si se requiere ARCore Extensions o no
  - definir estrategia concreta de marker tracking y re-acquisition
- stack de networking exacto:
  - Netcode for GameObjects + Unity Transport, u otra opcion
  - estrategia de sesion LAN local
- contrato de autoridad:
  - que calcula el host
  - que predice el cliente
  - que se reconcilia y como
- contrato de referencia espacial:
  - reglas del marcador
  - tolerancia jugable
  - reanclaje y rescan
- contrato de fallo:
  - tracking loss
  - host loss
  - reconnect/rejoin
  - app background/foreground
- contrato de rendimiento:
  - FPS floor y target
  - frame-time budget
  - thermal tolerance
  - tiers de dispositivo
- estructura de proyecto:
  - escenas
  - carpetas
  - assemblies
  - separacion de sistemas
- telemetria y profiling:
  - FPS
  - GC
  - RTT
  - jitter
  - drift espacial
- politica de versiones:

    - pinning de editor
    - pinning de paquetes criticos
    - politica de upgrades durante MVP

Technical Validation Gate

Antes de considerar cerrada esta seleccion de motor, debe ejecutarse un gate tecnico minimo:

- build funcional en al menos 2 dispositivos Android con ARCore
- validacion basica de tracking en condiciones reales
- validacion termica y de FPS en sesion corta
- confirmacion de compatibilidad entre editor y paquetes AR criticos

AI Tooling Posture

Optional MCP: MCP Unity (CoderGamester)

Se registra como herramienta auxiliar potencial, no como dependencia arquitectonica ni del camino critico de implementacion.

Postura recomendada:

- no usar MCP como pilar del arranque
- mantener un flujo de desarrollo completamente funcional sin MCP
- evaluarlo despues del primer vertical slice jugable si reduce trabajo repetitivo de editor

Motivo:
Aunque el repositorio esta activo y bien mantenido, sigue siendo tooling externo y no debe convertirse en punto de fallo para el MVP.

Optional Tooling Note

MCP Unity (CoderGamester)

- estado observado: activo
- requisitos: Unity 6+, Node.js 18+, npm 9+
- valor: inspeccion y manipulacion de escenas, GameObjects, componentes, materiales y tests
- postura: opcional, no bloqueante

No seleccionado como parte inicial del flujo

- Context7
- Unity MCP (CoplayDev)

Pueden reevaluarse mas adelante si el proyecto gana complejidad o si aparece dolor real en documentacion o automatizacion.

Architectural Decisions

Decision Summary

Category	Decision	Version	Rationale
Rendering Pipeline	URP	17.6	Mejor ajuste para AR movil Android, rendimiento y look estilizado del MVP
Networking Stack	Netcode for GameObjects + Unity Transport	NGO 2.11 / UT 6.5	Encaja mejor con topologia host-authoritative local y reduce riesgo frente a alternativas mas pesadas
State Management	State machine + servicios	N/A	Da control claro sobre setup, lobby, calibracion, carrera, pausa, recovery y rematch
Scene Structure	Bootstrap + Lobby + Race	N/A	Separa responsabilidades y simplifica arranque, sesion y depuracion
UI Framework	UGUI	Built-in	Mas maduro y practico para HUD AR camera-first y overlays del MVP
Persistence	Configuracion local minima	N/A	El MVP no necesita progresion compleja; solo ajustes y preferencias
Asset Loading	Scene-based sin Addressables	N/A	Suficiente para una pista MVP y evita complejidad temprana
Audio Architecture	Unity Audio + Mixer	Built-in	Cubre bien countdown, feedback, meta y control basico de mezcla
AR Stack	AR Foundation + ARCore XR Plugin	6.5 / 6.5	Base suficiente y oficial para Android AR con marker tracking
Surface Delimitation	1 marcador primario + varios secundarios de borde	N/A	Permite fijar origen/orientacion y mejorar validacion de la superficie jugable
Surface Calibration Model	Calibracion durante setup y geometria congelada antes de correr	N/A	Protege la confianza perceptual y evita que la pista cambie durante la carrera
Map Scaling	Escala validada segun superficie detectada, no recalculo continuo en runtime	N/A	La pista puede adaptarse a la mesa, pero debe quedar estable antes de iniciar la carrera
Authority Model	Host autoritativo; cliente solo envia input	N/A	Mantiene una unica verdad de carrera y reduce inconsistencias
Host Failure Policy	Terminar sesion y volver a lobby	N/A	Es la opcion mas fiable y controlable para MVP
Device Support Policy	Android ARCore general como objetivo; soporte oficial por tiers validados	N/A	Mantiene ambicion amplia, pero con una promesa de compatibilidad realmente testeable

Category	Decision	Version	Rationale
Version Policy	Pin total, sin upgrades salvo bloqueo critico	N/A	Minimiza drift tecnico durante el MVP

State Management

Approach: State machine + servicios

La aplicacion se organizara alrededor de una maquina de estados explicita para controlar el ciclo completo de sesion:

- boot
- permisos y validacion de dispositivo
- deteccion y calibracion AR
- lobby y union de jugadores
- carrera activa
- pausa o interrupcion
- recovery por tracking/network
- resultados y rematch

Los servicios encapsulan capacidades transversales:

- SessionService
- ARCalibrationService
- RaceSimulationService
- NetworkSessionService
- AudioService
- TelemetryService

Este enfoque evita singleton managers sin estructura y hace mas legible el contrato entre UX, AR y red.

Data Persistence

Save System: configuracion local minima

El MVP solo persistira:

- preferencias basicas
- volumen y feedback haptico
- flags de onboarding o ayudas
- ultimo perfil de configuracion local si aporta valor de UX

No se implementa progresion persistente, cloud save ni esquema complejo de partida guardada en esta fase.

Networking

Approach: NGO + Unity Transport con host authority

Contrato:

- el host crea y gobierna la sesion
- el cliente envia solo input de aceleracion y eventos de intencion
- el host resuelve estado de carrera, vuelta, penalizaciones, countdown y resultado
- la red no sincroniza mundo AR crudo; sincroniza estado de carrera y estados de sesion
- si el host falla, la sesion termina y ambos vuelven al lobby

No se implementa host migration en el MVP.

AR Calibration & Surface Delimitation

Approach: AR Foundation + ARCore XR Plugin con calibracion multi-marcador durante setup

Contrato espacial:

- un marcador primario fija origen, orientacion base y referencia principal
- varios marcadores secundarios de borde ayudan a validar la superficie disponible durante setup
- el sistema calcula un area valida de juego antes de empezar la carrera
- la pista se adapta a la superficie detectada solo durante setup y luego queda congelada antes de entrar a Race
- los marcadores secundarios mejoran la confianza de calibracion, pero no deben ser una dependencia activa del runtime de carrera
- si faltan marcadores criticos o la confianza espacial cae por debajo del umbral antes de la confirmacion, la sesion no entra en estado race-ready

Implicaciones:

- habra que definir visibilidad minima y numero minimo de marcadores secundarios utiles
- se deben fijar tolerancias geometricas aceptables para area, orientacion y alineacion entre dispositivos
- la estrategia de rescan debe ocurrir en setup o recovery, no reescalar la pista durante carrera
- debe validarse consistencia entre dispositivos para evitar que ambos vean escalas u orientaciones distintas

Scene Structure

Approach: Bootstrap + Lobby + Race

- Bootstrap inicializa servicios, validacion tecnica, politicas de version y telemetria
- Lobby gestiona creacion/union de sesion, readiness y preparacion de calibracion
- Race concentra runtime AR, simulacion, HUD y recovery de carrera

Esto deja una separacion clara entre ciclo de producto, sesion y gameplay.

UI Framework

Approach: UGUI

UGUI sera la base del MVP para:

- HUD de carrera
- countdown
- indicadores de tracking/red
- overlays de calibracion
- lobby y resultados

Se mantiene la regla UX de zona AR central protegida y controles perifericos en landscape.

Asset Management

Approach: carga por escena sin Addressables

- los recursos del MVP se cargan por escena
- no habra content streaming remoto en esta fase
- se evita complejidad de catalogos, etiquetas y dependencia temprana de Addressables

Si el proyecto crece a multiples pistas o contenido descargable, esta decision se revisa despues.

Audio Architecture

Approach: Unity Audio + Mixer

Arquitectura simple con buses minimos:

- UI
- SFX carrera
- feedback critico

Suficiente para countdown, meta, error de tracking, confirmaciones y senales arcade del MVP.

Validation Targets

Para que estas decisiones sean verificables, la arquitectura asume que el proyecto definira y medira como minimo:

- tiempo a mesa lista para jugar
- error maximo de alineacion y escala entre dispositivos
- tiempo maximo de recuperacion ante perdida de marcador en setup o recovery
- FPS sostenido por tier de dispositivo
- degradacion termica aceptable en sesiones repetidas
- tasa de sesiones invalidadas por perdida de confianza espacial

Architecture Decision Records

ADR-01: Se adopta Unity 6.3 LTS con paquetes oficiales alineados a movil/AR.

ADR-02: La sesion multijugador sera host-authoritative y la red solo sincronizara estado de carrera y sesion.

ADR-03: La referencia espacial del MVP se basa en un marcador primario y marcadores secundarios usados para calibracion de superficie durante setup.

ADR-04: La pista puede adaptarse al area detectada, pero su geometria queda congelada antes de iniciar la carrera.

ADR-05: La compatibilidad se expresa como objetivo Android ARCore general, pero el soporte oficial se define por tiers validados y matriz real de dispositivos.

ADR-06: El MVP operara con pinning estricto de versiones para evitar drift durante prototipado y validacion.

Cross-cutting Concerns

These patterns apply to ALL systems and must be followed by every implementation.

Cross-cutting State Authority

Before defining individual concerns, all systems must respect the same authority model:

- **Host authoritative:** session lifecycle, runtime config snapshot, frozen race geometry, race state, validated player input, results
- **Local-only authority:** rendering, local camera presentation, ephemeral UI state, temporary AR observations before calibration lock
- **Shared rule:** no system may mutate race-critical state after `TrackFreeze` unless the host explicitly authorizes it

Canonical high-level state machine: `Boot -> DeviceCheck -> ARSetup -> TableCalibration -> TrackFreeze -> SessionSync -> Countdown -> Race -> Results -> RematchOrExit`

Every subsystem must declare:

- in which states it may operate
- which events it may emit
- what it does when the session becomes invalid

Error Handling

Strategy: Result objects for recoverable boundary flows + global handler for unexpected failures

Use Rules:

- `Result` is mandatory at system boundaries and async/pre-race flows:
  - device validation
  - AR setup/calibration
  - session create/join
  - runtime config resolution
  - local persistence
  - scene/bootstrap transitions
- `Result` must NOT be used as the main control mechanism in hot paths:
  - `Update`
  - `FixedUpdate`
  - per-tick race simulation
  - per-frame AR presentation
- Exceptions are allowed for unexpected programmer/runtime faults only
- Global handler converts unhandled failures into safe state transitions, never into silent continuation

Error Shape (minimum):

- `code`
- `severity`
- `subsystem`
- `retryable`
- `isUserVisible`
- `correlationId`
- `recoveryHint`
- `message`

Severity Levels:

- **Critical:** current flow/session cannot continue safely
- **Recoverable:** feature can degrade or retry safely
- **Transient:** short-lived failure, retry allowed
- **DeveloperError:** invalid setup/contract violation; fail fast in dev, safe fallback in release

Recovery Policy:

- During setup, systems may retry or request user correction
- After `TrackFreeze`, structural inconsistency must pause, invalidate, or terminate session
- No silent fallbacks are allowed for race-critical failures
- No empty `catch` blocks are allowed

Example:

```
public enum ErrorSeverity
{
    Critical,
    Recoverable,
    Transient,
    DeveloperError
}

public readonly struct OperationResult
{
    public bool Success { get; }
    public string Code { get; }
    public ErrorSeverity Severity { get; }
    public string Subsystem { get; }
    public bool Retryable { get; }
    public bool IsUserVisible { get; }
    public string CorrelationId { get; }
    public string RecoveryHint { get; }
    public string Message { get; }

    private OperationResult(
        bool success,
        string code,
        ErrorSeverity severity,
        string subsystem,
        bool retryable,
        bool isUserVisible,
        string correlationId,
        string recoveryHint,
        string message)
    {
        Success = success;
        Code = code;
        Severity = severity;
        Subsystem = subsystem;
        Retryable = retryable;
        IsUserVisible = isUserVisible;
        CorrelationId = correlationId;
        RecoveryHint = recoveryHint;
        Message = message;
    }

    public static OperationResult Ok() =>
        new(true, "OK", ErrorSeverity.Recoverable, "SYSTEM", false, false, string.Empty, string.Empty, string.Empty);

    public static OperationResult Fail(
        string code,
        ErrorSeverity severity,
        string subsystem,
        bool retryable,
        bool isUserVisible,
        string correlationId,
        string recoveryHint,
        string message) =>
        new(false, code, severity, subsystem, retryable, isUserVisible, correlationId, recoveryHint, message);
}
```

Logging

Format: Unity logger wrapper with normalized categories, levels, and correlation fields
Destination: Unity Console in editor/dev builds; optional diagnostic dump/export in development builds only

Category Convention:

- `BOOT.System`
- `AR.Session`
- `AR.Calibration`
- `NET.Session`
- `NET.Transport`
- `RACE.Flow`
- `RACE.Simulation`
- `UI.HUD`
- `PERF.Runtime`

Mandatory Fields for relevant logs:

- `timestamp`
- `category`
- `level`
- `message`
- `sessionId`
- `matchId`
- `peerId`
- `phase`
- `correlationId`

Rules:

- No per-frame logging in hot paths
- No string formatting or allocation-heavy logging in race-critical loops
- `ERROR` and `WARN` are allowed in release-capable builds
- `DEBUG` is dev-only
- `TRACE` is temporary and forbidden in normal runtime
- Sensitive payloads, large object dumps, or unnecessary device identifiers must not be logged

Must Always Be Logged:

- session creation/join/leave/fail
- calibration start/confirm/fail/reset
- `TrackFreeze`
- countdown start/cancel
- race finish/invalidation
- host disconnect
- critical recovery transitions

Example:

```
public static class GameLog
{
    public static void Info(string category, string message, string sessionId, string matchId, string peerId, string phase, string correla
    {
        Debug.Log($"[{category}][INFO][session:{sessionId}][match:{matchId}][peer:{peerId}][phase:{phase}][corr:{correlationId}] {message}
    }

    public static void Warn(string category, string message, string sessionId, string matchId, string peerId, string phase, string correla
    {
        Debug.LogWarning($"[{category}][WARN][session:{sessionId}][match:{matchId}][peer:{peerId}][phase:{phase}][corr:{correlationId}] {m
    }

    public static void Error(string category, string message, string sessionId, string matchId, string peerId, string phase, string correl
    {
        Debug.LogError($"[{category}][ERROR][session:{sessionId}][match:{matchId}][peer:{peerId}][phase:{phase}][corr:{correlationId}] {me
    }
}
```

Configuration

Approach: ScriptableObjects + technical constants + local user persistence + runtime session snapshot

Ownership Rules:

- **Technical constants:** invariant technical limits and compile-time guards
- **ScriptableObjects:** balancing values, track profiles, AR thresholds, platform tiers, debug flags
- **PlayerPrefs:** simple user preferences only
- **JSON local:** structured local snapshots only when PlayerPrefs is insufficient
- **Runtime session snapshot:** resolved effective config used by a live session; host owns it and replicates it to clients

Precedence:

1. Technical constants
2. ScriptableObject defaults
3. Platform/tier adjustments
4. Local user preferences
5. Host-resolved runtime session snapshot

Rules:

- No race-critical simulation value may diverge between host and client
- Any value affecting race geometry, session rules, thresholds, or gameplay must be part of the runtime snapshot
- PlayerPrefs must not store critical gameplay or network authority data
- Persisted config must support basic versioning/migration
- Each config type must have a single owner and write path

Suggested Structure:

- `Assets/Data/Config/GameConstants/`
- `Assets/Data/Config/Balancing/`
- `Assets/Data/Config/Platform/`
- `Assets/Data/Config/Debug/`

Event System

Pattern: Typed event bus for inter-system notifications only

Allowed Uses:

- cross-system state notifications
- UI updates from domain state changes
- AR/session/race lifecycle broadcasts
- dev/debug observation hooks

Forbidden Uses:

- commands
- request/response flows
- network tick transport
- core gameplay simulation loop
- per-frame signaling

Rules:

- Events must be typed and immutable
- Stringly-typed events are forbidden
- Main-thread synchronous dispatch by default
- Event ordering must be deterministic where required by session flow
- Subscriptions must be explicitly cleaned up
- Direct method calls or C# events are preferred for tightly coupled local interactions
- If an event affects race/session flow, ownership and allowed source must be explicit

Naming Convention:

- `CalibrationLocked`
- `TrackFreezeCompleted`
- `PeerDisconnected`
- `RaceCountdownStarted`
- `RaceInvalidated`

Example:

```
public readonly struct TrackFreezeCompleted
{
    public string SessionId { get; }
    public string CorrelationId { get; }

    public TrackFreezeCompleted(string sessionId, string correlationId)
    {
        SessionId = sessionId;
        CorrelationId = correlationId;
    }
}

public interface IEventBus
{
    void Publish<TEvent>(TEvent eventData);
    void Subscribe<TEvent>(Action<TEvent> handler);
    void Unsubscribe<TEvent>(Action<TEvent> handler);
}
```

Debug Tools

Available Tools:

- runtime overlay for FPS, frame time, GC, CPU/GPU frame indicators
- AR panel showing tracking state, calibration confidence, primary marker presence, secondary marker count, drift estimate
- network panel showing role, RTT, jitter, packet loss, disconnect state
- session panel showing canonical state machine state, snapshot version, race status
- dev commands for:
  - reset AR session
  - invalidate calibration
  - simulate host disconnect
  - simulate packet loss/jitter
  - return to lobby
  - export session diagnostic snapshot

Gating:

- enabled only in `UNITY_EDITOR` or `DEVELOPMENT_BUILD`
- mutating commands restricted to development contexts
- session-affecting commands should be host-only when relevant
- no debug command may remain player-accessible in release

Rules:

- overlays should be read-only by default
- refresh rate should be throttled, not per-frame if unnecessary
- debug tools must not meaningfully affect race performance
- debug tools must support AR, network, and recovery validation, not just generic engine metrics

Fault Injection & Observability

All critical systems must support testable failure injection and observable state transitions.

Required Injectables:

- AR tracking loss
- marker loss during setup
- host disconnect
- client disconnect
- packet loss/jitter
- app pause/resume
- delayed calibration confirmation

Required Session Snapshot Fields:

- build identifier
- device model
- OS version
- Unity version
- AR package versions
- networking package versions
- session snapshot version
- tier classification
- active thresholds
- correlationId/sessionId/matchId

Validation Metrics

The architecture assumes the project will measure at minimum:

AR

- time to usable tracking
- time to stable calibration
- anchor drift over time
- world alignment error between devices
- tracking loss rate
- recovery success rate during setup

Network

- RTT p50/p95/p99
- jitter
- packet loss
- dropped/duplicated critical events
- session join success rate
- disconnect recovery outcome

Performance

- FPS p50/p95
- frame-time spikes
- GC alloc/frame
- memory peak
- thermal degradation indicators

Anti-patterns

The following are forbidden:

- silent fallback on race-critical failures
- empty `catch` blocks
- stringly-typed events
- logging in hot paths by default
- multiple mutable sources of truth for race configuration
- using event bus for commands or per-frame simulation
- debug tools exposed in release builds

Project Structure

Organization Pattern

Pattern: Hybrid

Definition:
El proyecto usa una estructura hibrida con dos reglas simultaneas:

1. En `Assets/ProyectoMovimiento`, los recursos se organizan por tipo siguiendo convenciones de Unity: `Art`, `Audio`, `Data`, `Prefabs`, `Scenes`, `Scripts`, `Tests`.
2. Dentro de runtime code, la organizacion se hace por responsabilidad y dominio real: `Core`, `Runtime/App`, `Runtime/Infrastructure`, `Runtime/Features`, `Runtime/UI`, `Runtime/Debug`.

Rationale:
Esta mezcla mantiene orden natural para Unity sin convertir el proyecto en un arbol generico por tipos tecnicos. Tambien evita sobredescomponer el MVP en demasiadas micro-features. La estructura debe

responder a una pregunta simple: “esto es una regla compartida, una integracion tecnica, un flujo del juego, una vista o una herramienta de desarrollo?”

Directory Structure

```
Proyecto-Movimiento/
├── Assets/
│   └── ProyectoMovimiento/
│       ├── Art/
│       │   ├── Materials/
│       │   ├── Models/
│       │   ├── Shaders/
│       │   ├── Textures/
│       │   └── Vfx/
│       ├── Audio/
│       │   ├── Mixers/
│       │   ├── Music/
│       │   └── Sfx/
│       ├── Data/
│       │   ├── Config/
│       │   │   ├── Balancing/
│       │   │   ├── Debug/
│       │   │   ├── GameConstants/
│       │   │   └── Platform/
│       │   ├── MarkerProfiles/
│       │   └── Tracks/
│       ├── Prefabs/
│       │   ├── Calibration/
│       │   ├── Race/
│       │   ├── Results/
│       │   ├── Shared/
│       │   └── UI/
│       ├── Scenes/
│       │   ├── Boot.unity
│       │   ├── Lobby.unity
│       │   ├── Race.unity
│       │   └── Sandboxes/
│       │       ├── CalibrationSandbox.unity
│       │       ├── NetworkSandbox.unity
│       │       └── PerformanceSandbox.unity
│       ├── Scripts/
│       │   ├── Core/
│       │   │   ├── ProyectoMovimiento.Core.asmdef
│       │   │   ├── Contracts/
│       │   │   ├── Events/
│       │   │   ├── Logging/
│       │   │   ├── Results/
│       │   │   ├── State/
│       │   │   └── Timing/
│       │   ├── Runtime/
│       │   │   ├── ProyectoMovimiento.Runtime.asmdef
│       │   │   ├── App/
│       │   │   │   ├── Bootstrap/
│       │   │   │   └── Composition/
│       │   │   ├── Infrastructure/
│       │   │   │   ├── AR/
│       │   │   │   ├── Networking/
│       │   │   │   ├── Persistence/
│       │   │   │   ├── Platform/
│       │   │   │   └── Telemetry/
│       │   │   ├── Features/
│       │   │   │   ├── Calibration/
│       │   │   │   ├── Session/
│       │   │   │   ├── Race/
│       │   │   │   └── Results/
│       │   │   ├── UI/
│       │   │   │   ├── Hud/
│       │   │   │   ├── Presenters/
│       │   │   │   ├── Screens/
│       │   │   │   └── Widgets/
│       │   │   └── Debug/
│       │   │       └── Runtime/
│       │   └── Editor/
│       │       ├── ProyectoMovimiento.Editor.asmdef
│       │       ├── Build/
│       │       ├── Tools/
│       │       └── Validation/
│       └── Tests/
│           ├── EditMode/
│           │   ├── Fixtures/
│           │   ├── TestDoubles/
│           │   └── ProyectoMovimiento.Tests.EditMode.asmdef
```

```
│           └── PlayMode/
│               ├── Harnesses/
│               ├── Smoke/
│               └── ProyectoMovimiento.Tests.PlayMode.asmdef
├── Packages/
│   ├── manifest.json
│   └── packages-lock.json
├── ProjectSettings/
├── UserSettings/
├── docs/
└── _bmad-output/
```

Assembly Definition Policy

Allowed asmdefs for MVP:

- `ProyectoMovimiento.Core`
- `ProyectoMovimiento.Runtime`
- `ProyectoMovimiento.Editor`
- `ProyectoMovimiento.Tests.EditMode`
- `ProyectoMovimiento.Tests.PlayMode`

Rules:

- No se crea una asmdef nueva por feature en esta etapa.
- Solo se permite una asmdef adicional si aisla una dependencia externa concreta o resuelve un problema real de compilacion medido.
- `Core` puede ser referenciado por `Runtime`, `Editor` y `Tests`.
- `Runtime` no puede ser referenciado por `Core`.
- `Editor` nunca puede ser dependencia de runtime.
- `Tests` pueden depender de `Core` y `Runtime`, nunca al reves.

System Location Mapping

System	Location	Responsibility
App bootstrap	`Assets/ProyectoMovimiento/Scripts/Runtime/App/Bootstrap/`	Entrada de aplicacion, composition root y transicion al flujo inicial
Scene composition	`Assets/ProyectoMovimiento/Scripts/Runtime/App/Composition/`	Wiring por escena, installers y dependencias visibles
Device validation	`Assets/ProyectoMovimiento/Scripts/Runtime/Infrastructure/Platform/`	Compatibilidad ARCore, permisos y checks de dispositivo
AR runtime integration	`Assets/ProyectoMovimiento/Scripts/Runtime/Infrastructure/AR/`	Adaptadores de AR Foundation / ARCore y estado tecnico AR
Networking runtime integration	`Assets/ProyectoMovimiento/Scripts/Runtime/Infrastructure/Networking/`	NGO, Unity Transport, host/client lifecycle y adaptadores tecnicos
Persistence	`Assets/ProyectoMovimiento/Scripts/Runtime/Infrastructure/Persistence/`	PlayerPrefs, JSON local y snapshots runtime
Telemetry and profiling	`Assets/ProyectoMovimiento/Scripts/Runtime/Infrastructure/Telemetry/`	FPS, RTT, jitter, drift, recovery metrics
Shared contracts and primitives	`Assets/ProyectoMovimiento/Scripts/Core/`	Result types, state contracts, events, logging contracts y tipos compartidos
Calibration flow	`Assets/ProyectoMovimiento/Scripts/Runtime/Features/Calibration/`	Setup AR, validacion de marcadores, delimitacion de superficie y TrackFreeze

System	Location	Responsibility
Session flow	`Assets/ProyectoMovimiento/Scripts/Runtime/Features/Session/`	Lobby, create/join, ready state y countdown previo a carrera
Race flow	`Assets/ProyectoMovimiento/Scripts/Runtime/Features/Race/`	Simulacion, inputs validos, checkpoints, vueltas, invalidacion y flow activo
Results flow	`Assets/ProyectoMovimiento/Scripts/Runtime/Features/Results/`	Resultados, rematch y salida controlada
Screens and HUD	`Assets/ProyectoMovimiento/Scripts/Runtime/UI/`	Presentacion visual y binding de estado sin logica de dominio
Runtime debug	`Assets/ProyectoMovimiento/Scripts/Runtime/Debug/Runtime/`	Overlays, paneles y comandos solo para desarrollo
Editor-only tooling	`Assets/ProyectoMovimiento/Scripts/Editor/`	Validadores, utilidades de build y herramientas de editor
Config assets	`Assets/ProyectoMovimiento/Data/Config/`	ScriptableObjects de tuning, platform tiers y flags de debug
Track definitions	`Assets/ProyectoMovimiento/Data/Tracks/`	Definiciones de pista y escalas validadas
Marker profiles	`Assets/ProyectoMovimiento/Data/MarkerProfiles/`	Perfiles y tolerancias de marcadores
Productive scenes	`Assets/ProyectoMovimiento/Scenes/`	Boot, Lobby y Race

System	Location	Responsibility
Technical sandboxes	`Assets/ProyectoMovimiento/Scenes/Sandboxes/`	Aislamiento tecnico para calibracion, red y rendimiento
EditMode tests	`Assets/ProyectoMovimiento/Tests/EditMode/`	Logica pura, contratos, configuracion y utilidades
PlayMode tests	`Assets/ProyectoMovimiento/Tests/PlayMode/`	Smoke tests, integracion de escenas, harnesses AR/red

Classification Examples

Correct classification examples:

- `LapTimeRules` -> `Scripts/Core/State/`
- `ARSessionAdapter` -> `Scripts/Runtime/Infrastructure/AR/`
- `RaceCoordinator` -> `Scripts/Runtime/Features/Race/`
- `CountdownPresenter` -> `Scripts/Runtime/UI/Presenters/`
- `RaceDebugPanel` -> `Scripts/Runtime/Debug/Runtime/`

Invalid classification examples:

- `RaceCountdownPresenter` in `Core`
- `ARAnchorRepository` in `UI`
- `VehicleSpawnerMono` in `Core`
- production logic inside `Sandboxes`

Naming Conventions

Files

- Scripts: `PascalCase`
  - `RaceCoordinator.cs`
  - `CalibrationLocked.cs`
- Scenes: `PascalCase`
  - `Boot.unity`
  - `CalibrationSandbox.unity`
- Prefabs: `PascalCase`
  - `RaceHud`
  - `PrimaryMarkerAnchor`
- ScriptableObjects: `PascalCase` with semantic suffix
  - `RaceConfig`
  - `MarkerProfile`
  - `DeviceTierConfig`

Code Elements

Element	Convention	Example
Namespace	Match folder intent	`ProyectoMovimiento.Runtime.Features.Race`
Class	`PascalCase`	`RaceCoordinator`
Interface	`IPascalCase`	`IEventBus`
Method	`PascalCase`	`TryLockCalibration`
Public Property	`PascalCase`	`SessionId`
Private Field	`_camelCase`	`_currentState`
Parameter / Local	`camelCase`	`peerId`
Enum	`PascalCase`	`RaceState`
Enum Member	`PascalCase`	`Countdown`
Constant	`PascalCase`	`DefaultCountdownSeconds`

Asset Rules

- Materials use `Mat` suffix
  - `TrackPreviewMat`
- Audio clips use semantic suffix
  - `CountdownBeepSfx`
  - `LobbyLoopMusic`
- Sprites/textures use semantic suffix
  - `RaceHudFrameSprite`
  - `PrimaryMarkerTexture`
- Events use completed-state or domain action naming
  - `CalibrationLocked`
  - `TrackFreezeCompleted`
  - `RaceInvalidated`

Scene Rules

- `Boot` only boots the app, validates global prerequisites and routes to the correct next state.
- `Lobby` owns session creation/join, readiness and pre-race coordination.
- `Race` owns active runtime composition for race flow only.
- Productive scenes must have explicit composition roots.
- No productive scene may depend on hidden hierarchy assumptions or `GameObject.Find`.
- Sandboxes must stay out of production build flow unless explicitly promoted.

Sandbox Rules

- Sandboxes exist only for isolated technical verification.
- Sandboxes must not be referenced by production code or productive scene composition.
- Sandboxes may use debug helpers, mocks and harnesses freely.
- A sandbox must have one technical purpose only:
  - calibration
  - networking
  - performance
- If a sandbox becomes part of normal flow, it must be promoted or removed.

Debug and Editor Rules

- `Runtime/Debug` contains runtime-only development helpers gated by `UNITY_EDITOR` or `DEVELOPMENT_BUILD`.
- `Scripts/Editor` contains editor-only tooling, validators and build helpers.
- `Debug` is never a production dependency.
- Anything under `Editor` must never compile into player runtime.

Architectural Boundaries

- `Core` is almost pure C#. By default, it does not reference `UnityEngine`, AR Foundation or NGO.
- `Infrastructure` implements technical adapters and engine/package integrations. It does not own race rules.
- `Features` orchestrates game flow and authoritative use cases.
- `UI` presents state and emits UI intent; it does not decide gameplay rules.
- Shared code moves to `Core` only if it is truly cross-feature and not package-specific.
- If code touches AR Foundation, Unity Transport, NGO, scene objects or platform APIs, it does not belong in `Core`.
- Every script must be classifiable into exactly one area.
- Production code must never depend on sandboxes.
- New folders or layers require a concrete ownership reason, not aesthetic preference.

Implementation Patterns

These patterns ensure consistent implementation across all AI agents.

Novel Patterns

Multi-Marker Calibration + TrackFreeze Pattern

Purpose:
Transformar observaciones espaciales inestables de setup en una referencia de carrera estable, validada y congelada antes del inicio competitivo.

Scope:
Aplica solo durante setup y recovery guiado. No opera libremente durante carrera activa.

Owner:
Host session authority, con apoyo de servicios locales de AR/calibracion.

When It Applies:

- setup inicial de sesion
- re-calibracion autorizada
- recovery espacial tras perdida critica de confianza

When It Does Not Apply:

- carrera normal en estado estable
- ajustes visuales locales del cliente
- sincronizacion por frame de gameplay

Source of Truth:

- durante calibracion: el host consolida la mejor interpretacion valida
- despues de `TrackFreeze`: el snapshot congelado es la unica verdad espacial compartida

Core Rules:

- el marcador primario es obligatorio
- los secundarios son opcionales, pero pueden mejorar validacion y escala
- no se acepta freeze sin preview estable y confirmacion explicita del host
- tras `TrackFreeze`, ningun cliente reinterpreta por su cuenta la geometria compartida
- un recovery no puede promover una nueva geometria sin una nueva confirmacion explicita

Minimum Acceptance Criteria:

- marcador primario visible y coherente
- superficie valida dentro de tolerancias configuradas
- confianza minima alcanzada
- preview estable durante una ventana corta definida por configuracion
- sin conflicto geometrico critico entre marcadores observados

State Model:

- `Scanning`
- `PreviewReady`
- `FreezePending`
- `Frozen`
- `Invalid`

Components:

- `PrimaryMarkerTracker`
- `SecondaryMarkerTracker`
- `SurfaceValidationService`
- `CalibrationPreviewBuilder`
- `TrackFreezeCoordinator`
- `CalibrationReportBuilder`

Implementation Guide:

```
public enum CalibrationState
{
    Scanning,
    PreviewReady,
    FreezePending,
    Frozen,
    Invalid
}

public readonly record struct CalibrationReport(
    string CorrelationId,
    bool PrimaryMarkerVisible,
    int SecondaryMarkerCount,
    float Confidence,
    string DecisionReason,
    bool Accepted);

public sealed class TrackFreezeCoordinator
{
    public OperationResult<RaceSpaceSnapshot> TryConfirmFreeze(
        CalibrationPreview preview,
        bool hostConfirmed,
        float minimumConfidence,
        int nextVersion,
        string trackScaleProfile,
        string correlationId)
    {
        if (!hostConfirmed)
        {
            return OperationResult<RaceSpaceSnapshot>.Fail(
                code: "TRACK_FREEZE_NOT_CONFIRMED",
                severity: ErrorSeverity.Recoverable,
                subsystem: "AR.Calibration",
                retryable: true,
                isUserVisible: true,
                correlationId: correlationId,
                recoveryHint: "Esperar confirmacion del host.",
                message: "El host aun no confirma el freeze.");
        }

        if (!preview.IsStable || !preview.SurfaceIsValid || preview.Confidence < minimumConfidence)
        {
            return OperationResult<RaceSpaceSnapshot>.Fail(
                code: "TRACK_FREEZE_PREVIEW_INVALID",
                severity: ErrorSeverity.Recoverable,
                subsystem: "AR.Calibration",
                retryable: true,
                isUserVisible: true,
                correlationId: correlationId,
                recoveryHint: "Revisar marcadores y estabilidad de superficie.",
                message: "La preview no cumple las condiciones minimas para congelar.");
        }

        var snapshot = new RaceSpaceSnapshot(
            nextVersion,
            preview.Origin,
            preview.Rotation,
            preview.SurfaceBounds,
            trackScaleProfile,
            preview.Confidence,
            correlationId);

        return OperationResult<RaceSpaceSnapshot>.Ok(snapshot);
    }
}
```

Valid Examples:

- confirmar freeze con marcador primario valido y secundarios consistentes
- rechazar freeze si la preview pierde estabilidad antes de la confirmacion

Invalid Examples:

- recalcular el track durante carrera solo porque reaparecio un marcador secundario
- aceptar freeze con cualquier marcador visible sin validar consistencia espacial

Race Space Snapshot Pattern

Purpose:
Definir un contrato espacial compartido, versionado e inmutable que el host publica y el cliente aplica de forma atomica antes de la carrera o tras recovery autorizado.

Scope:
Sincronizacion de geometria compartida y configuracion espacial estable. No contiene estado efimero por frame.

Owner:
Host.

Consumers:

- cliente remoto
- flow de sesion
- flow de carrera
- UI dependiente del estado espacial validado

When It Applies:

- despues de `TrackFreeze`
- despues de recovery que termina en nuevo freeze confirmado

When It Does Not Apply:

- correcciones menores locales de render
- simulacion de coche por frame
- actualizacion continua de tracking crudo

Snapshot Must Include:

- version
- origen
- orientacion
- limites validos de superficie
- escala o perfil final de pista
- confianza de calibracion
- correlationId

Snapshot Must Not Include:

- referencias a GameObjects runtime
- transforms efimeros por frame
- estado de fisica viva
- observaciones crudas no consolidadas

Core Rules:

- solo el host produce snapshots validos
- el cliente nunca reconstruye por inferencia su propia version equivalente
- la aplicacion del snapshot es atomica
- una nueva version solo nace tras un nuevo freeze confirmado
- snapshots obsoletos o incompatibles deben rechazarse explicitamente

Implementation Guide:

```
public readonly record struct RaceSpaceSnapshot(
    int Version,
    Vector3 Origin,
    Quaternion Rotation,
    Bounds SurfaceBounds,
    string TrackScaleProfile,
    float Confidence,
    string CorrelationId);

public sealed class RaceSpaceSnapshotApplier
{
    private int _appliedVersion = -1;

    public OperationResult Apply(RaceSpaceSnapshot snapshot)
    {
        if (snapshot.Version <= _appliedVersion)
        {
            return OperationResult.Fail(
                code: "RACE_SPACE_SNAPSHOT_STALE",
                severity: ErrorSeverity.Recoverable,
                subsystem: "NET.Session",
                retryable: false,
                isUserVisible: false,
                correlationId: snapshot.CorrelationId,
                recoveryHint: "Ignorar snapshot obsoleto.",
                message: "La version del snapshot ya fue aplicada o fue superada.");
        }

        _appliedVersion = snapshot.Version;
        return OperationResult.Ok();
    }
}
```

Valid Examples:

- aplicar snapshot final antes de pasar a countdown
- rechazar snapshot con version anterior o payload invalido

Invalid Examples:

- mezclar world space y local space sin contrato explicito
- permitir que varios sistemas muten el snapshot por conveniencia

Spatial Trust / Recovery Pattern

Purpose:
Traducir calidad espacial percibida en una senal operativa clara que controle warning, pausa, recovery e invalidacion de sesion.

Scope:
Opera despues de `TrackFreeze` y durante carrera o recovery.

Owner:
El host decide la reaccion de sesion. Cada cliente solo reporta senales locales.

Simplified MVP State Model:

- `Trusted`
- `Degraded`
- `Lost`

Meaning:

- `Trusted`: carrera normal
- `Degraded`: warning visual y monitoreo reforzado, sin reinterpretar geometria
- `Lost`: pausa host-driven y entrada a recovery guiado

Inputs:

- estado de tracking AR local
- visibilidad/coherencia del marcador primario
- diferencia respecto al snapshot congelado
- senales de alineacion o deriva compartidas por la sesion

Core Rules:

- degradacion menor no puede reconfigurar la pista
- perdida critica no puede ignorarse con solo warning visual indefinido
- el sistema usa histeresis o ventana temporal para evitar oscilaciones rapidas
- solo el host decide si la sesion continua, pausa, entra en recovery o se invalida
- si recovery no recupera confianza suficiente, la sesion se invalida

Implementation Guide:

```csharp
public enum SpatialTrustState
{
    Trusted,
    Degraded,
    Lost
}

public readonly record struct SpatialTrustSnapshot(
    SpatialTrustState State,
    float Confidence,
    float DriftMeters,
    bool PrimaryMarkerVisible,
    string CorrelationId);

public sealed class SpatialTrustPolicy
{
    public SpatialTrustState Evaluate(
        float confidence,
        bool primaryMarkerVisible,
        float driftMeters,
        float degradedThreshold,
        float lostThreshold,
        float maxDriftMeters)
    {
        if (!primaryMarkerVisible || confidence < lostThreshold || driftMeters > maxDriftMeters)
        {
            return SpatialTrustState.Lost;
        }

        if (confidence < degradedThreshold)
        {
            return SpatialTrustState.Degraded;
        }

        return SpatialTrustState.Trusted;
    }
}
```

Valid Examples:

- mostrar warning en `Degraded` sin cambiar la geometria compartida
- pausar y guiar reacquisicion del marcador primario al entrar en `Lost`

Invalid Examples:

- seguir corriendo indefinidamente en `Lost`
- recuperar confianza con una sola muestra estable sin histeresis

Communication Patterns

Pattern: Hibrido: referencias directas para colaboradores cercanos + eventos tipados entre sistemas

Purpose:
Mantener claridad local sin volver opaco el sistema completo.

Rules:

- referencias directas dentro de una misma feature o agregado pequeno
- eventos tipados solo para hechos de dominio o cambios importantes entre sistemas
- los eventos representan hechos, no comandos disfrazados
- no se publican eventos por frame o por tick de gameplay
- payloads de eventos deben ser pequenos e inmutables

Use Direct Reference For:

- controller a service local
- presenter a view
- composicion cerrada dentro de una feature

Use Event For:

- `CalibrationLocked`
- `TrackFreezeCompleted`
- `RaceStarted`
- `RecoveryEntered`
- `PeerDisconnected`

Invalid Uses:

- reemplazar una llamada local simple por un evento
- encadenar eventos como flujo principal de control
- publicar eventos de posicion o UI por frame

Example:

```
public sealed class CalibrationFlowController : MonoBehaviour
{
    [SerializeField] private CalibrationView _view;

    private CalibrationService _calibrationService;
    private IEventBus _eventBus;

    public void Construct(CalibrationService calibrationService, IEventBus eventBus)
    {
        _calibrationService = calibrationService;
        _eventBus = eventBus;
    }

    public void ConfirmFreeze()
    {
        var result = _calibrationService.TryConfirmFreeze();

        if (!result.Success)
        {
            _view.ShowError(result.Message);
            return;
        }

        _eventBus.Publish(new TrackFreezeCompleted(result.Value.Version, result.Value.Confidence));
    }
}
```

Entity Patterns

Creation: Prefabs + factories ligeras; pooling solo donde profiling lo justifique

Definition of Lightweight Factory:

- instancia prefab
- hace wiring minimo
- aplica configuracion de creacion
- no toma decisiones de negocio
- no orquesta flujo de sesion o carrera

Rules:

- gameplay no instancia entidades arbitrariamente sin pasar por un punto de creacion claro
- pooling queda fuera del MVP salvo evidencia de profiling
- si entra pooling, debe respetar contratos explicitos de reset y lifecycle

Invalid Uses:

- factory con reglas de gameplay
- factory que resuelve dependencias globales arbitrarias
- pooling introducido sin evidencia de frame spikes

Example:

```
public sealed class RaceCarFactory
{
    private readonly RaceCar _raceCarPrefab;

    public RaceCarFactory(RaceCar raceCarPrefab)
    {
        _raceCarPrefab = raceCarPrefab;
    }

    public RaceCar Create(Vector3 position, Quaternion rotation, Transform parent)
    {
        return Object.Instantiate(_raceCarPrefab, position, rotation, parent);
    }
}
```

State Patterns

Pattern: State machines explicitas para flujos grandes; estados simples para entidades acotadas

Use Formal State Machine When:

- hay transiciones explicitas
- hay recovery paths
- hay timeouts o guards
- varios sistemas reaccionan al mismo estado

Use Simple State When:

- la entidad tiene pocos estados
- no hay flujo complejo de recovery
- no hay varios subsistemas coordinados

Rules:

- flujos grandes deben definir transiciones validas e invalidas
- guards y timeouts deben ser explicitos
- no duplicar autoridad de estado entre UI, entidad y coordinador central

Invalid Uses:

- state machine completa para cada entidad efimera
- modelar recovery complejo solo con booleans
- duplicar el mismo estado en varios sistemas con autoridad ambigua

Example:

```
public enum SessionState
{
    Boot,
    Lobby,
    Calibration,
    TrackFrozen,
    Countdown,
    Race,
    Results
}

public sealed class SessionStateMachine
{
    public SessionState CurrentState { get; private set; } = SessionState.Boot;

    public bool TryTransition(SessionState nextState)
    {
        if (CurrentState == SessionState.Calibration && nextState == SessionState.TrackFrozen)
        {
            CurrentState = nextState;
            return true;
        }

        if (CurrentState == SessionState.TrackFrozen && nextState == SessionState.Countdown)
        {
            CurrentState = nextState;
            return true;
        }

        return false;
    }
}
```

Data Patterns

Access: ScriptableObjects de autoria + servicios/repositorios runtime

Rules:

- ScriptableObjects son autoria/configuracion, no estado mutable de sesion
- servicios coordinan acceso, no reemplazan ownership de datos
- repositorios runtime son duenos del acceso a estado operativo o configuracion efectiva
- assets de diseno no se mutan en runtime
- snapshots de sesion contienen configuracion efectiva cuando afecta la verdad compartida

Invalid Uses:

- escribir progreso o confianza espacial dentro de ScriptableObjects
- usar un servicio global como contenedor generico de cualquier dato
- abrir escritura libre a varios sistemas sobre el mismo repositorio

Example:

```
[CreateAssetMenu(menuName = "ProyectoMovimiento/Race Config")]
public sealed class RaceConfig : ScriptableObject
{
    [SerializeField] private float _defaultLapTimeLimit = 60f;
    [SerializeField] private string _defaultTrackScaleProfile = "Medium";

    public float DefaultLapTimeLimit => _defaultLapTimeLimit;
    public string DefaultTrackScaleProfile => _defaultTrackScaleProfile;
}

public sealed class RaceConfigRepository
{
    private readonly RaceConfig _raceConfig;

    public RaceConfigRepository(RaceConfig raceConfig)
    {
        _raceConfig = raceConfig;
    }

    public string GetDefaultTrackScaleProfile() => _raceConfig.DefaultTrackScaleProfile;
}
```

Consistency Rules

Pattern	Convention	Enforcement
TrackFreeze	Preview estable + confirmacion explicita del host	La carrera no inicia sin freeze valido
Race Space Snapshot	Inmutable, versionado, host-owned, aplicacion atomica	Rechazo de snapshots obsoletos o incompatibles
Spatial Trust	`Trusted / Degraded / Lost` con histeresis y host authority	Warning menor, pausa/recovery en perdida critica
Communication	Referencia directa local, evento tipado entre sistemas	No eventos por frame ni comandos disfrazados
Entity Creation	Prefab + factory ligera; pooling solo con profiling	No logica de negocio en factories
State	FSM para flujos grandes, estado simple para entidades pequenas	Guards y transiciones invalidas explicitas
Data	ScriptableObjects para autoria, runtime services/repos para estado vivo	Prohibido mutar assets de diseno en runtime

Architecture Validation

Validation Summary

Check	Result	Notes
Decision Compatibility	PASS	Unity 6.3 LTS, URP, AR Foundation, ARCore, NGO y Unity Transport son coherentes con los patrones, cross-cutting y estructura definidos
GDD Coverage	PASS	El MVP queda cubierto de extremo a extremo: setup, calibracion, sesion local, carrera, resultados, recovery, HUD y rendimiento
Pattern Completeness	PASS	Hay patrones definidos para comunicacion, creacion de entidades, estado, acceso a datos, errores, eventos y los 3 patrones novelosos clave
Epic Mapping	PASS	No existe todavia artifact formal de epics, pero la arquitectura si mapea correctamente las features MVP: Boot, Calibration, Session, Race y Results
Document Completeness	PASS	Estan presentes motor/stack, tabla de decisiones, cross-cutting, estructura, naming, patrones y no quedan placeholders/TODO/TBD

Coverage Report

Systems Covered: 8/8
Patterns Defined: 7
Decisions Made: 16

Validation Notes

- No se encontraron conflictos bloqueantes entre stack, patrones, cross-cutting y estructura.
- El documento no contiene placeholders ni texto provisional pendiente.
- Existe una diferencia no bloqueante con el GDD original, que aun habla de un marcador unico; la arquitectura ya refino esa decision a marcador primario mas secundarios opcionales durante setup.

Issues Resolved

- La estructura del proyecto se compacto para evitar sobrefragmentacion temprana.
- Los patrones novelosos y estandar se endurecieron con alcance, ownership, ejemplos validos e invalidos.
- La validacion confirmo cobertura arquitectonica para setup, calibracion, carrera, recovery, HUD, networking local y rendimiento minimo.

Validation Date

2026-04-16

Development Environment

Prerequisites

- Unity Hub with **Unity 6.3 LTS** installed
- Android Build Support with **SDK**, **NDK** and **OpenJDK**
- 2 ARCore-compatible Android phones for multiplayer validation
- Primary reference device: **Samsung S24 FE**
- Local Wi-Fi or hotspot for host-client testing
- Git and VS Code for source control and editor workflow
- **Node.js 18+** and **npm 9+** if optional MCP Unity tooling is enabled

AI Tooling (MCP Servers)

The following MCP server was selected during architecture as optional, non-blocking tooling:

MCP Server	Purpose	Install Type
MCP Unity (CoderGamester)	Optional Unity scene/editor inspection and AI-assisted development support	Node.js-based external MCP server

Setup:

1. Install **Node.js 18+** and **npm 9+**.
2. Configure **MCP Unity (CoderGamester)** after the first Unity project open, following the repository instructions.
3. Keep the project fully operable without MCP; treat it as an accelerator, not a dependency.

Setup Commands

```
# Create a new clean Unity project in Unity Hub using Unity 6.3 LTS
# Add Android Build Support (SDK/NDK/OpenJDK) from Unity Hub
# Open the project and pin these packages in Package Manager:
#   com.unity.render-pipelines.universal@17.6
#   com.unity.xr.arfoundation@6.5
#   com.unity.xr.arcore@6.5
#   com.unity.netcode.gameobjects@2.11
#   com.unity.transport@6.5
# Then create the initial scenes: Boot, Lobby, Race
```

First Steps

1. Create the clean Unity project and switch target platform to Android.
2. Add and pin the required Unity packages defined by the architecture.
3. Create the root folder structure under `Assets/ProyectoMovimiento` and the initial asmdefs.
4. Configure MCP Unity only if it reduces real editor friction after the first vertical slice.
5. Implement the minimum vertical slice: marker detection, stable preview, TrackFreeze, one car and one acceleration button.